



**BSc. Hons in Ethical Hacking & Scybersecurity**  
**ST4017CMD Introduction to Programming**  
**Softwarica College of IT & E-Commerce**  
**Submission Date: July 2026**

**Submit by:**  
**Chandan Sah**  
**Student ID: 17618858**

**Submitted to:**  
**Prabisha Khadka**

## Table of Contents

|                                                                    |    |
|--------------------------------------------------------------------|----|
| LIST OF FIGURES .....                                              | 3  |
| ABSTRACT.....                                                      | 4  |
| CHAPTER 1: INTRODUCTION .....                                      | 4  |
| 1.1 Background.....                                                | 4  |
| 1.2 Problem Statement .....                                        | 4  |
| 1.3 Objectives .....                                               | 5  |
| 1.4 Scope.....                                                     | 5  |
| CHAPTER 2: LITERATURE REVIEW .....                                 | 5  |
| 2.1 Commercial and Open-Source Threat Intelligence Platforms ..... | 5  |
| 2.2 IOC Enrichment Data Sources .....                              | 6  |
| 2.3 Graph-Based Threat Visualisation .....                         | 6  |
| 2.4 Credential Security in Security Tooling.....                   | 6  |
| CHAPTER 3: SYSTEM DESIGN AND ARCHITECTURE .....                    | 7  |
| 3.1 Overall Architecture.....                                      | 7  |
| 3.2 Technology Stack.....                                          | 8  |
| 3.3 Database Design.....                                           | 9  |
| CHAPTER 4: IMPLEMENTATION.....                                     | 10 |
| 4.1 Configuration and Credential Security .....                    | 10 |
| 4.2 IOC Lookup and Risk Scoring Engine .....                       | 12 |
| 4.3 Report Generation.....                                         | 13 |
| 4.4 Threat Relationship Graph.....                                 | 13 |
| 4.5 Scan History and Search .....                                  | 14 |
| CHAPTER 5: RESULTS AND EVALUATION .....                            | 15 |
| 5.1 Dashboard Overview .....                                       | 15 |
| 5.2 IOC Lookup in Practice .....                                   | 16 |
| 5.3 Analytics .....                                                | 17 |
| 5.4 Network Graph.....                                             | 18 |
| 5.5 Scan History and Audit Trail .....                             | 19 |
| 5.6 Discussion.....                                                | 20 |
| CHAPTER 6: SECURITY CONSIDERATIONS .....                           | 20 |
| CHAPTER 7: CONCLUSION AND FUTURE WORK .....                        | 21 |
| REFERENCES .....                                                   | 22 |

## LIST OF FIGURES

|                                                                                                          |    |
|----------------------------------------------------------------------------------------------------------|----|
| Figure 1: main.py — application entry point. ....                                                        | 7  |
| Figure 2: requirements.txt — third-party dependencies used by the project. ....                          | 9  |
| Figure 3: db_manager.py — SQLite schema definition, insert, and search logic.....                        | 10 |
| Figure 4: config.py — Fernet-based encryption of stored API credentials. ....                            | 11 |
| Figure 5: setup.py — first-run configuration script for encrypting and storing API credentials. ....     | 12 |
| Figure 6: ioc_lookup.py — orchestrating VirusTotal and AbuseIPDB lookups and persisting the result. .... | 12 |
| Figure 7: report_generator.py — CSV and PDF export logic built on pandas and ReportLab. ....             | 13 |
| Figure 8: graph_viz.py — building the entity and threat relationship graph with NetworkX. ....           | 14 |
| Figure 9: scan_history.py — retrieval and search wrapper over the database layer. ....                   | 15 |
| Figure 10: SOC Engine dashboard, showing summary statistics, threat distribution, and recent activity..  | 16 |
| Figure 11: IOC Investigation Module showing a live lookup and combined risk assessment. ....             | 17 |
| Figure 12: Risk score distribution histogram across recorded scans. ....                                 | 18 |
| Figure 13: Interactive threat relationship graph for a single investigated event. ....                   | 19 |
| Figure 14: Scan Audit Logs and History, showing the full record of past investigations. ....             | 20 |

## ABSTRACT

Soc analysts may waste valuation time checking the same Indicator of Compromise (IOC) across different threat intelligence platforms. In this thesis, the author presents SOC Engine, a python desktop application to simplify an analyst's threat hunting activities. SOC Engine fetches feeds from Virus Total and AbusedIPDB and generates an overall risk assessment score. The application also comes with an interface built on Tkinter, an encrypted configuration file, an audit database powered by SQLite, built-in analytics capabilities, network visualization and reports to PDF/CSV. Finally, by helping to reduce the repetitive tasks associated with the correlation of threat intelligence from multiple resources and helping SOC analysts to be more efficient, SOC Engine acts as an IOC lookup consolidator. This thesis describes the design and implementation of SOC Engine, assessment, limitations and future improvement suggestions.

## CHAPTER 1: INTRODUCTION

### 1.1 Background

Every organization that has exposure to the Internet deals with the suspicion of certain activity, be it an irregular login attempt, an unknown IP address, or a dubious file hash. SOC analysts waste a lot of time investigating potential threats and various indicators of compromise (IOCs) in order to understand if they pose a real danger or not. However, even though there are platforms that help with checking if an IOC is malicious or not, such as VirusTotal and AbuseIPDB, they only provide a small amount of information regarding the IOC. It makes the job of SOC analysts even harder since they need to conduct research on multiple platforms to get sufficient information about the IOC. This is why SOC Engine was created. IT is a platform that allows users to benefit multiple sources of information in one place, thus making IOR analysis faster and less resource-consuming.

### 1.2 Problem Statement

As the number of security alerts grows, checking IOCs can take a significant amount of time, especially they need to be evaluated against multiple threats intelligence sources. Since every sources uses a different interface, the analyst can come to different conclusions depending on which source they use. Additionally, performing research in a browser is not always reliable or trackable, especially if further context or history about an IOC is needed at a later point. This is why SOC engine is

necessary: it allows analysts to perform IOC analysis faster, more consistently, and more traceable manner compared to traditional methods.

### 1.3 Objectives

- To develop a desktop application to query multiple threat intelligence API's from a single text box.
- To integrate VirusTotal and AbuseIPDB information in a single understandable risk score and classification.
- To stick each lookup in a local database where it can be later searched, reviewed and exported.
- To display a severity histogram. Displaying a severity histogram (SEVERITY HISTOGRAM):
- To ensure that API credentials are stored securely, even though they are stored in plain text.

### 1.4 Scope

Compared to the previous version, the SOC engine v4.0 is designed to use primarily for IP address lookups. It should be mentioned that the program's design can easily be adapted to include domain names and files hashes in the future. The tool is suggested to be used by one person or small team of analysts to work on one computer, whereas VirusTotal and AbuseIPDB are the only two threat intelligence sources available for now, which will be expanded in the future versions, described in Chapter 7.

## CHAPTER 2: LITERATURE REVIEW

### 2.1 Commercial and Open-Source Threat Intelligence Platforms

Threat intelligence enrichment is included in a range of much more comprehensive capabilities in commercial SIEM solutions like Splunk, IBM QRadar, and Microsoft Sentinel, which are costly, and are often more than a student or small team needs to understand how to correlate threats. On the other end of the spectrum, scripts designed for a specific purpose that feature only a single API are unlikely to store

data, aggregate data sources, or present results in an effective way. SOC Engine is right in the middle of these two extremes: focused enough to build and reason about within a single project, but structured enough to aggregate multiple sources and maintain a permanent record, neither of which ends of the spectrum is successful alone.

## 2.2 IOC Enrichment Data Sources

VirusTotal provides the analysis results from various antivirus and URL scanners for detecting whether a certain resource was marked as malicious or benign([VirusTotal,2026](#)). AbuseIPDB, in its turn, allows to collect the reports made by different network operators, which establish the badness score for IPs on the basis of occurred incidents. It is necessary to use the data from both services to calculate the accurate risk score for a threat, which is why the SOC engine uses the intelligence provided by these platform([AbuseIPDB,2026](#)). As we see, the motivation to design the risk-scoring engine described in Chapter 4 is to combine different sources of threat intelligence, which allows to obtain a more accurate result than using only one of them.

## 2.3 Graph-Based Threat Visualisation

Graph-based visualization allows one to better understanding the relationship between security events and indicators than presenting the information in a list. SOC engine relies on the Python [NetworkX](#) library, and [Matplotlib \(Hagberg et al., 2008; Hunter, 2007\)](#), is a lightweight way to build such a graph without a dedicated graph database an approach SOC Engine.

## 2.4 Credential Security in Security Tooling

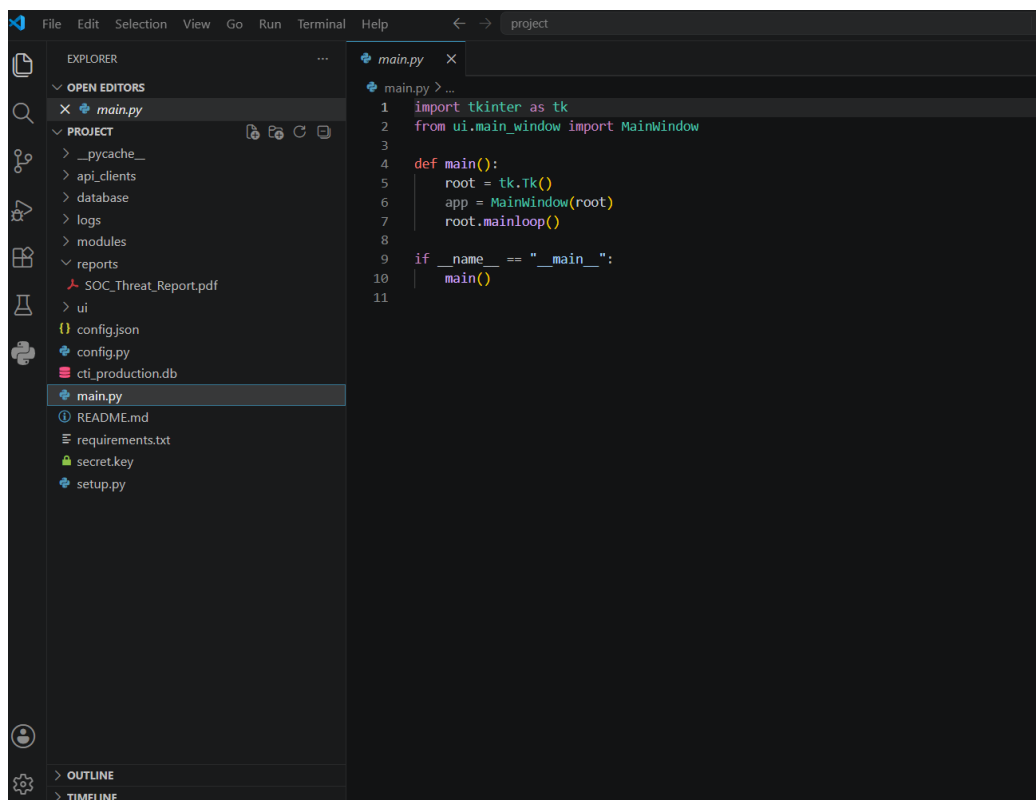
Protecting API keys should be a priority for any application that relies on online threat intelligence services to protect itself. Therefore, it is not safe to store them as plain text, and SOC engine uses the cryptography library's Fernet implementation

to store them in an encrypted for enhanced security and ease of use ([Python Software Foundation, 2026](#)).

## CHAPTER 3: SYSTEM DESIGN AND ARCHITECTURE

### 3.1 Overall Architecture

The layered architecture of SOC Engine helps maintain the application structure and makes it easy to maintain. The Tkinter interface gives the user input and separate modules are available for IOC lookups, risk scoring, scan history, visualisation and reporting. There are specific client modules to process API requests to VirusTotal and AbuseIPDB and all scan results are stored in a SQLite database. Securely manage API keys with an encrypted configuration module. The application begins with a basic entrypoint that brings up the main window and separates the app startup code from application logic.



*Figure 1: main.py — application entry point.*

In the early stages of development, prior to implementing the layered architecture, the risk-scoring procedure was placed directly in the button callback, which made testing and maintenance considerably more difficult. The refactoring stage involved extracting that procedure into a separate module.

### 3.2 Technology Stack

SOC Engine was built using a limited set of reliable Python tools, listed in the requirement.txt and presented in Figure 2 below. In particular, Requests library was used to send API requests to VirusTotal and AbuseIPDB, Matplotlib- to build scan analytics visualisation [matplotlib \(Hunter, 2007\)](#), and [reportlab \(ReportLab, 2026\)](#) for PDF export, [pandas \(McKinney, 2010\)](#) for CSV export, [networkx \(Hagberg et al., 2008\)](#) for the relationship graph, and [cryptography \(Python Software Foundation, 2026\)](#) for encrypting stored keys. No web framework or external database server was used; the interface runs entirely on Tkinter, keeping the tool installable without administrator rights.



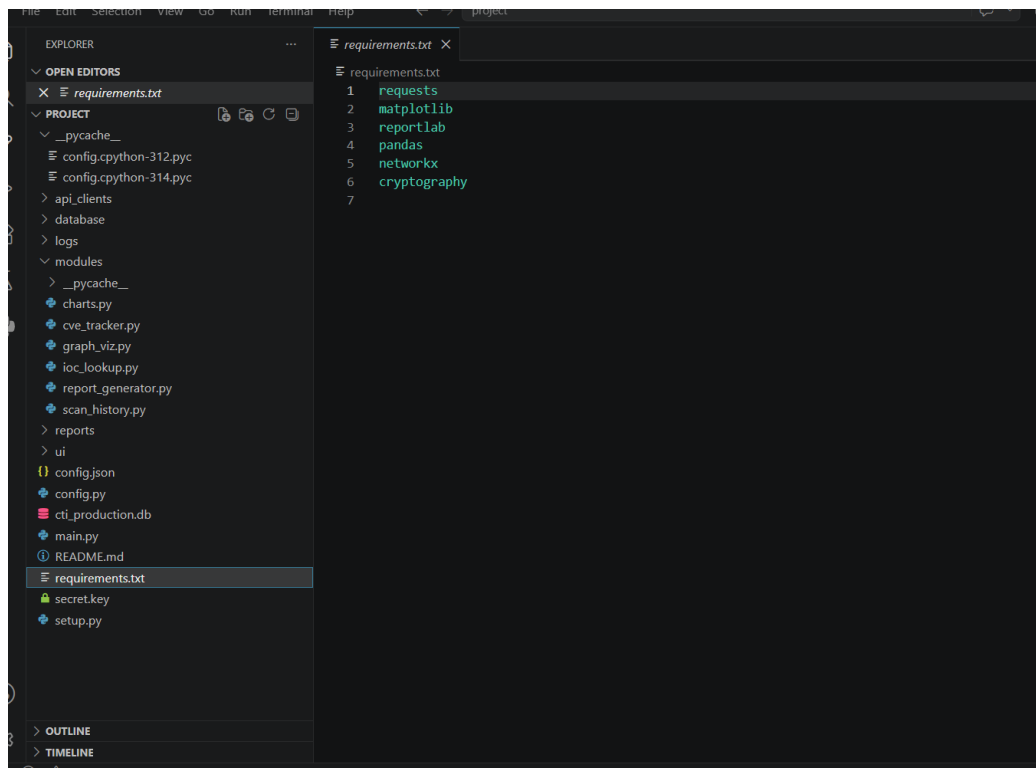


Figure 2: requirements.txt — third-party dependencies used by the project.

### 3.3 Database Design

All of the scan results are stored in a local SQLite database (cti\_production.db) maintained by the DBManager class (SOC Engine). The Scan History table (scan\_history) contains the scan ID, date and time when the scan was started, details about the type of the IOC and its value, risk score, original API response, and user name of the analyst (default is admin). The database module is able to store new scan results and search through scan history, allowing quick retrieval of previously analysed results as seen in figure 8.

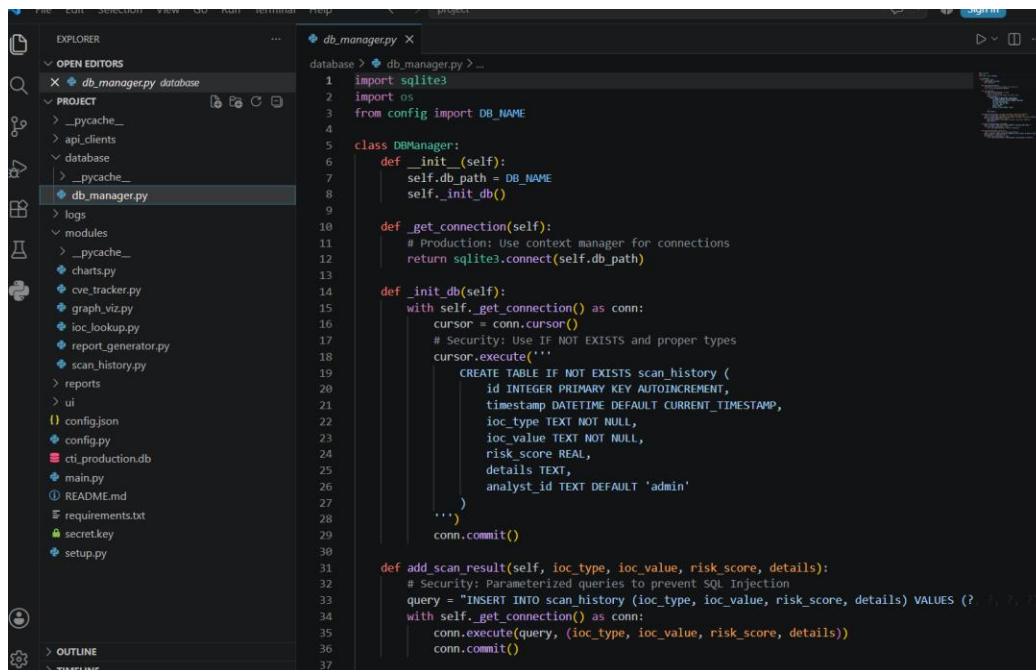


Figure 3: *db\_manager.py* — SQLite schema definition, insert, and search logic.

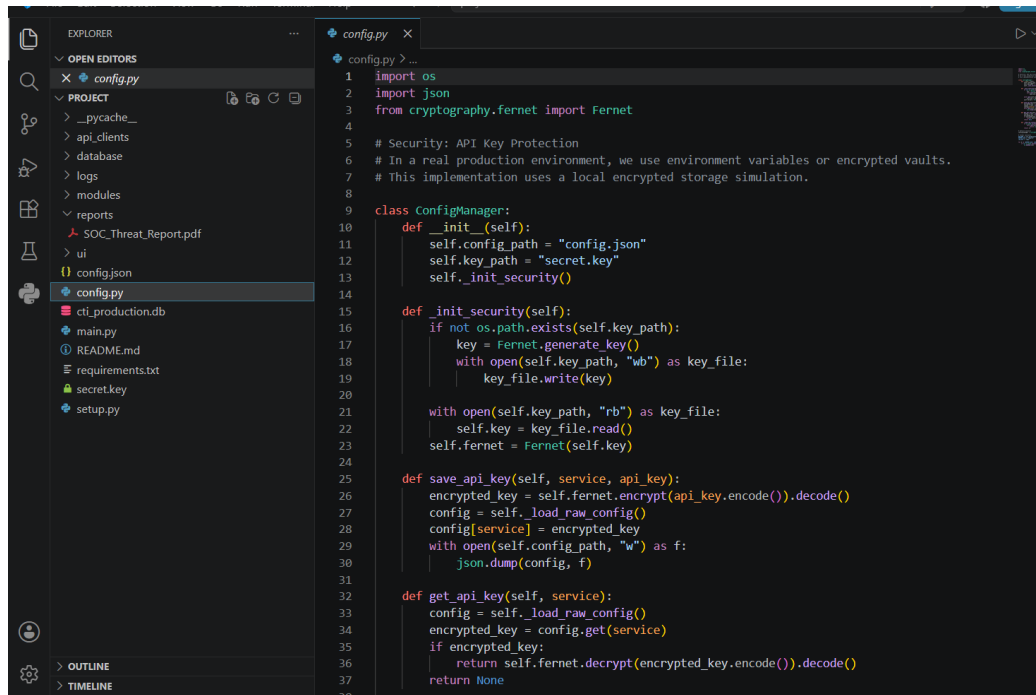
To enhance security, all database queries which include user input are parameterized to avoid the usage of string concatenation and prevent any possible SQL injection attempts. Even though SOC Engine is designed for single-user at a time, the search feature allows querying the database using the information extracted and stored from the API request details. This way, the application can be used to scan the website for phishing risks, and if any suspicious data is found, it can be searched later using words like “phishing” to find the corresponding request and analyze the result.

## CHAPTER 4: IMPLEMENTATION

### 4.1 Configuration and Credential Security

Since SOC Engine uses two API services, it is critical to make sure that their credentials are kept confidential and secure. The config module, shown in Figure 4, generates a [Fernet](#) encryption key on the first run and stores it in secret.key file. At the same time, the API keys in the encrypted form are saved in config.json . Such an approach allows to keep the encryption key and the encrypted data separate, which

is considered to be best practice to ensure confidentiality and security of the credentials to API services.



```
1 import os
2 import json
3 from cryptography.fernet import Fernet
4
5 # Security: API Key Protection
6 # In a real production environment, we use environment variables or encrypted vaults.
7 # This implementation uses a local encrypted storage simulation.
8
9 class ConfigManager:
10     def __init__(self):
11         self.config_path = "config.json"
12         self.key_path = "secret.key"
13         self._init_security()
14
15     def _init_security(self):
16         if not os.path.exists(self.key_path):
17             key = Fernet.generate_key()
18             with open(self.key_path, "wb") as key_file:
19                 key_file.write(key)
20
21         with open(self.key_path, "rb") as key_file:
22             self.key = key_file.read()
23         self.fernet = Fernet(self.key)
24
25     def save_api_key(self, service, api_key):
26         encrypted_key = self.fernet.encrypt(api_key.encode()).decode()
27         config = self._load_raw_config()
28         config[service] = encrypted_key
29         with open(self.config_path, "w") as f:
30             json.dump(config, f)
31
32     def get_api_key(self, service):
33         config = self._load_raw_config()
34         encrypted_key = config.get(service)
35         if encrypted_key:
36             return self.fernet.decrypt(encrypted_key.encode()).decode()
37         return None
```

Figure 4: *config.py* — Fernet-based encryption of stored API credentials.

The setup script shown below in Figure 5 is initiated during installation ask the user to enter their VirusTotal and AbuseIPDB API key. These key are then encrypted with the `confi_managet.save_api_key` function, and saved for future use, which finalizes the setup process.

```

1 from config import config_manager
2 import getpass
3
4 def setup():
5     print("--- SOC Engine Production Setup ---")
6     vt_key = input("Enter VirusTotal API Key: ")
7     abuse_key = input("Enter AbuseIPDB API Key: ")
8
9     config_manager.save_api_key("virustotal", vt_key)
10    config_manager.save_api_key("abuseipdb", abuse_key)
11
12    print("\n[+] Configuration encrypted and saved successfully.")
13    print("[+] Run 'python main.py' to start the dashboard.")
14
15 if __name__ == "__main__":
16     setup()
17

```

Figure 5: `setup.py` — first-run configuration script for encrypting and storing API credentials.

## 4.2 IOC Lookup and Risk Scoring Engine

The core of the application is the `IOCLookup` class. Its `lookup_ip` method, shown in Figure 6, performs all the necessary steps by [VirusTotal](#) and [AbuseIPDB](#) clients in turn, passing their responses to a `scores` function, writes the result to the database, and returns a Python dictionary the UI can render directly.

```

1 import json
2 from api_clients.virustotal_client import VirusTotalClient
3 from api_clients.abuseipdb_client import AbuseIPDBClient
4 from database.db_manager import DBManager
5
6 class IOCLookup:
7     def __init__(self):
8         self.vt_client = VirusTotalClient()
9         self.abuse_client = AbuseIPDBClient()
10        self.db_manager = DBManager()
11
12    def lookup_ip(self, ip_address):
13        vt_data = self.vt_client.get_ip_report(ip_address)
14        abuse_data = self.abuse_client.check_ip(ip_address)
15
16        risk_score = self.calculate_risk_score(vt_data, abuse_data)
17
18        details = {
19            "virustotal": vt_data,
20            "abuseipdb": abuse_data,
21        }
22
23        self.db_manager.add_scan_result("IP", ip_address, risk_score, json.dumps(details))
24
25    def return {
26        "ioc": ip_address,
27        "type": "IP",
28        "risk_score": risk_score,
29        "details": details
30    }
31
32    def calculate_risk_score(self, vt_data, abuse_data):
33        # Security-focused scoring:
34        # In a real SOC, even a few detections are critical.
35
36        vt_malicious = 0
37        if vt_data and "data" in vt_data:
38

```

Figure 6: `ioc_lookup.py` — orchestrating [VirusTotal](#) and [AbuseIPDB](#) lookups and persisting the result.

Scoring system is weighted towards being secure, not an average of all sources. For example, if VirusTotal has more than 5 malicious result or AbuseIPDB has more than 90% confidence, the risk score would be at least 85. If there is any malicious result on VirusTotal and a score of 25% or higher on AbuseIPDB, a lower but still substantial risk would be considered a score of at least 40. Otherwise, it would have the score of half of the percentage from AbuseIPDB, which I think is far, since the threat is not confirmed yet but still may exist if any of the sources are correct.

### 4.3 Report Generation

The scan result can be exported as CSV and PDF file so that analysts can share with other parties easily with SOC Engine. Figure 7 below shows the ReportGenerator that uses pandas ([pandas](#)) to generate CSV files and ReportLab ([ReportLab's](#)) to design PDF files with legible text and page breaks.

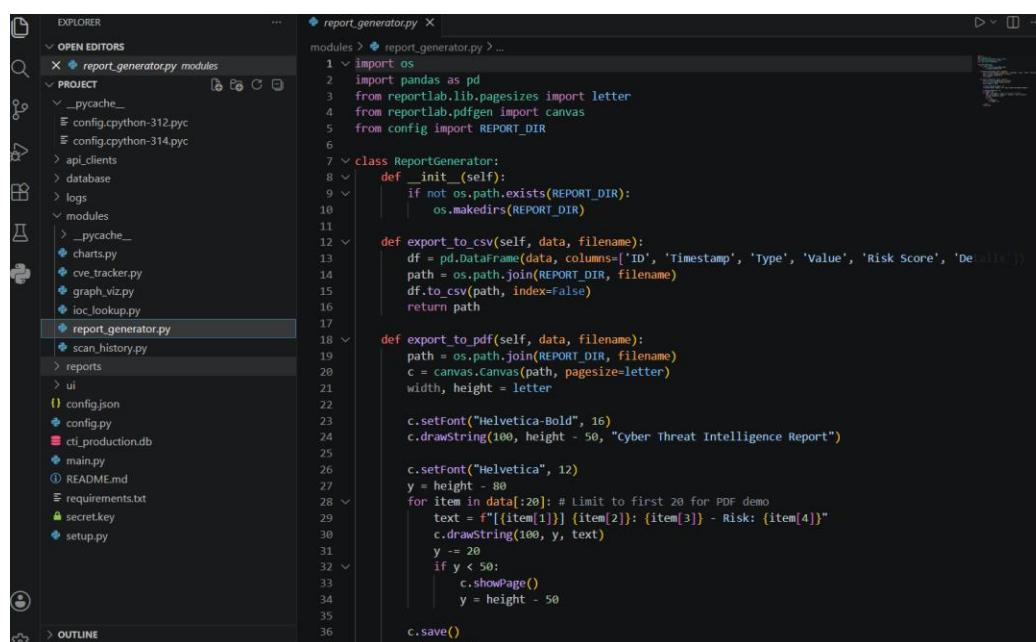
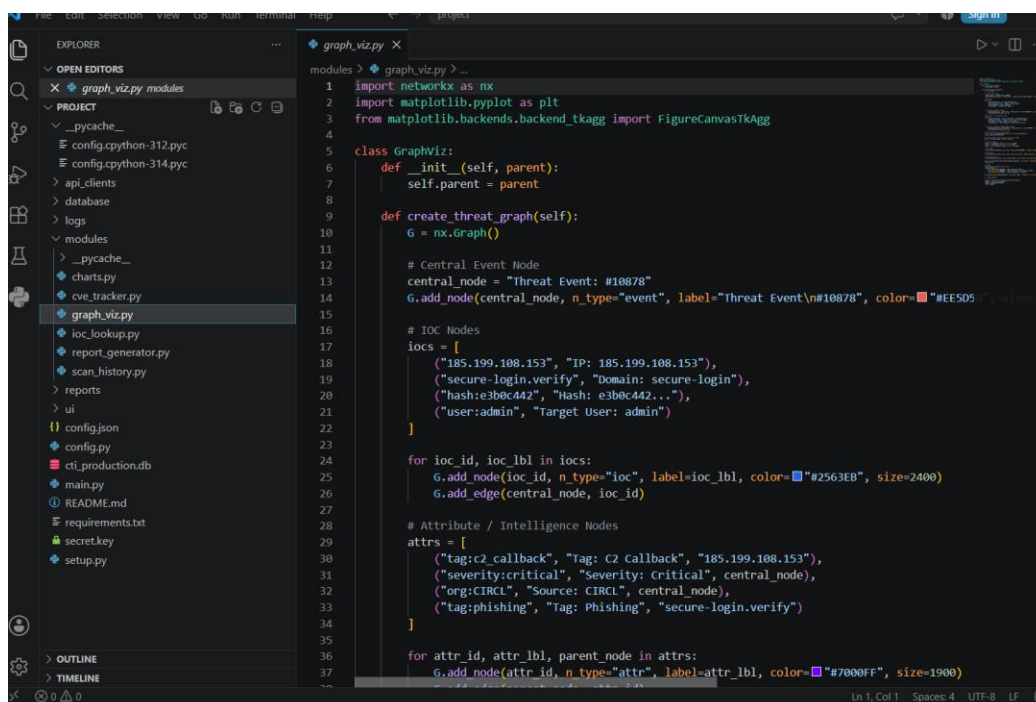


Figure 7: `report_generator.py` — CSV and PDF export logic built on pandas and ReportLab.

### 4.4 Threat Relationship Graph

The GraphViz module was used to visualize the basic network graph, relying on the NetworkX library for each identified security events. As can be seen in Figure 8, the

event was visualized as a central node with connections to other nodes that represent the related IOCs, such as IP addresses, domains, and file hashes, with their corresponding attributes, including tags, source, and severity. Such a visualization could help an analyst comprehend the relationships between discovered security events compared to reviewing the information presented in tabular form.



```
1 import networkx as nx
2 import matplotlib.pyplot as plt
3 from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg
4
5 class GraphViz:
6     def __init__(self, parent):
7         self.parent = parent
8
9     def create_threat_graph(self):
10        G = nx.Graph()
11
12        # Central Event Node
13        central_node = "Threat Event: #10878"
14        G.add_node(central_node, n_type="event", label="Threat Event\n#10878", color="#EE5D5D", size=2400)
15
16        # IOC Nodes
17        iocs = [
18            ("185.199.108.153", "IP: 185.199.108.153"),
19            ("secure-login.verify", "Domain: secure-login"),
20            ("hash:e3b0c442", "Hash: e3b0c442..."),
21            ("user:admin", "Target User: admin")
22        ]
23
24        for ioc_id, ioc_lbl in iocs:
25            G.add_node(ioc_id, n_type="ioc", label=ioc_lbl, color="#2563EB", size=2400)
26            G.add_edge(central_node, ioc_id)
27
28        # Attribute / Intelligence Nodes
29        attrs = [
30            ("tag:c2_callback", "Tag: C2 Callback", "185.199.108.153"),
31            ("severity:critical", "Severity: Critical", central_node),
32            ("org:CIRCL", "Source: CIRCL", central_node),
33            ("tag:phishing", "Tag: Phishing", "secure-login.verify")
34        ]
35
36        for attr_id, attr_lbl, parent_node in attrs:
37            G.add_node(attr_id, n_type="attr", label=attr_lbl, color="#7080FF", size=1900)
```

Figure 8: graph\_viz.py — building the entity and threat relationship graph with NetworkX.

## 4.5 Scan History and Search

The final scans are saved and can be retrieved through ScanHistory module. This is demonstrated as in Figure 9, users can view the complete scan history or scan history search with keywords. Further, it is possible to isolate it as a separate module so that it can be expanded later with new functionality, e.g., sorting the search result by data, by an analyst or by risk score, etc., without having to modify the database.

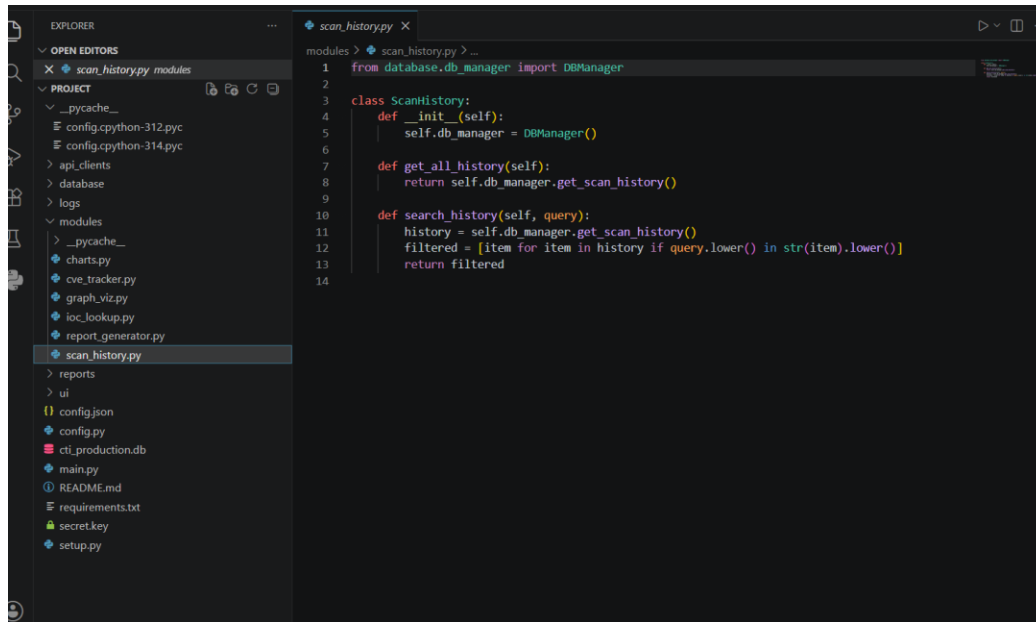


Figure 9: `scan_history.py` — retrieval and search wrapper over the database layer.

## CHAPTER 5: RESULTS AND EVALUATION

The designed application was tested through practical application of the program by conducting actual IP address lookups. The results demonstrated that the interface, database, and reports were processed correctly throughout the experiment.

### 5.1 Dashboard Overview

The home dashboard Figure 10 displays the system’s summary on four cards that shows an aggregate view of total threats, high-risk indicators of compromise (IOC), scans conducted, and active alerts with 24-hour trends. Furthermore, the dashboard includes a donut chart to see threat categories distribution, recent malicious activity and fast search IOCs.

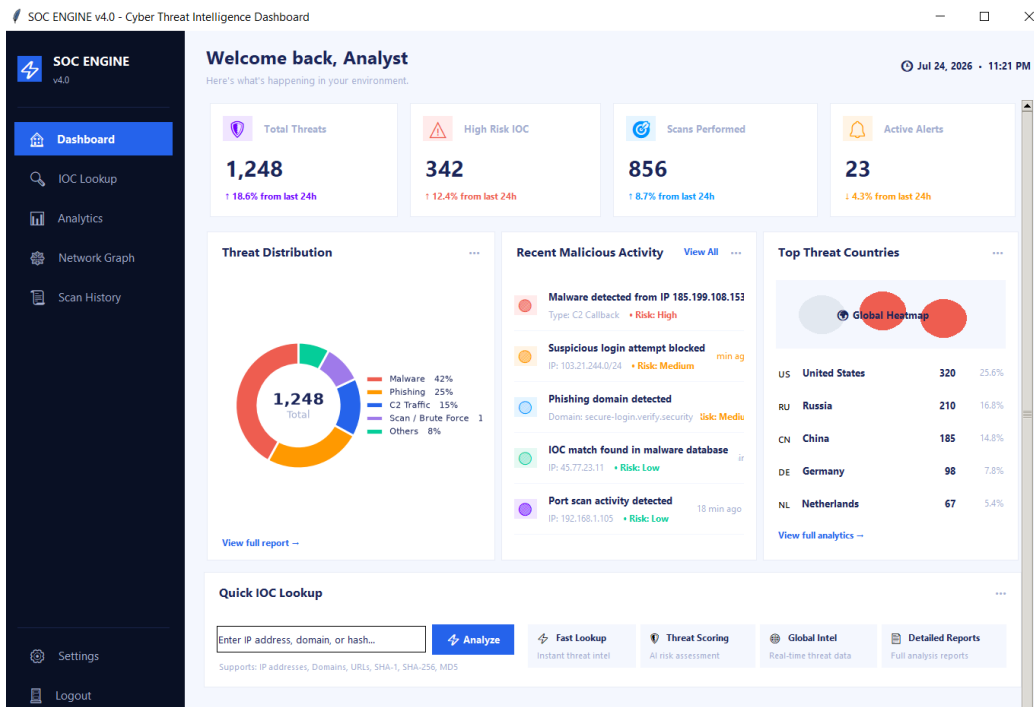
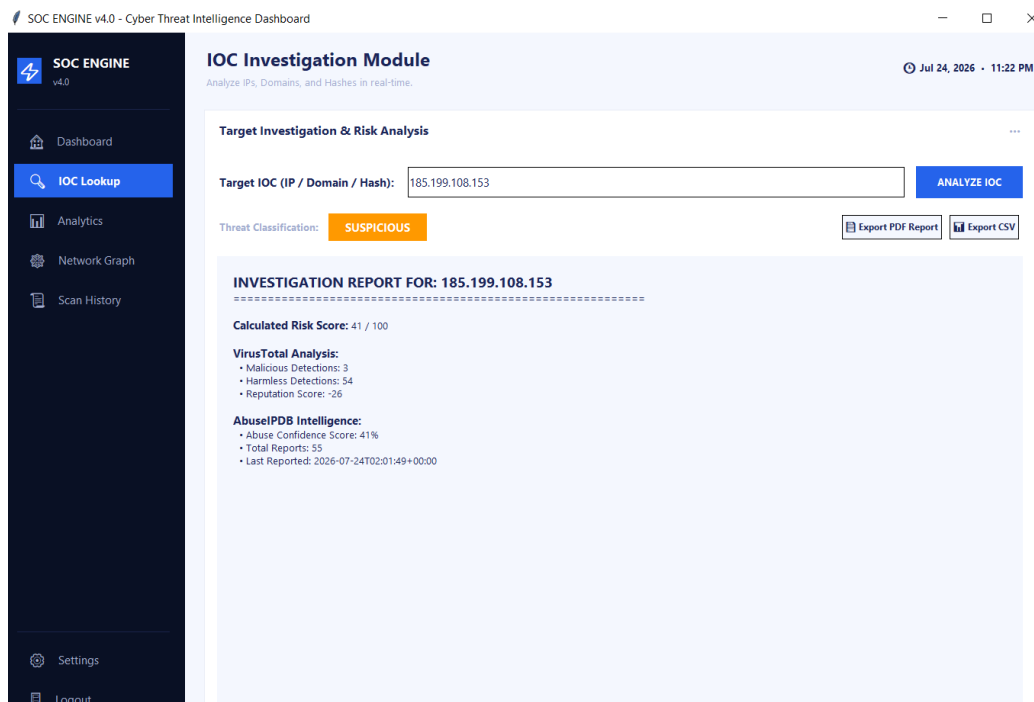


Figure 10: SOC Engine dashboard, showing summary statistics, threat distribution, and recent activity.

## 5.2 IOC Lookup in Practice

Figure 11 below shows an example of a real IOC lookup for IP Address 185.199.108.153. As we can see from the results below, the application scored a risk level of 41 or “Suspicious”. This would be in line with the instruction given in section 4.2 as there were 3 malicious programs found in VirusTotal and 55 reports of abuse in AbuseIPDB, which translates to a medium suspicious risk level on the IP Address.





*Figure 11: IOC Investigation Module showing a live lookup and combined risk assessment.*

### 5.3 Analytics

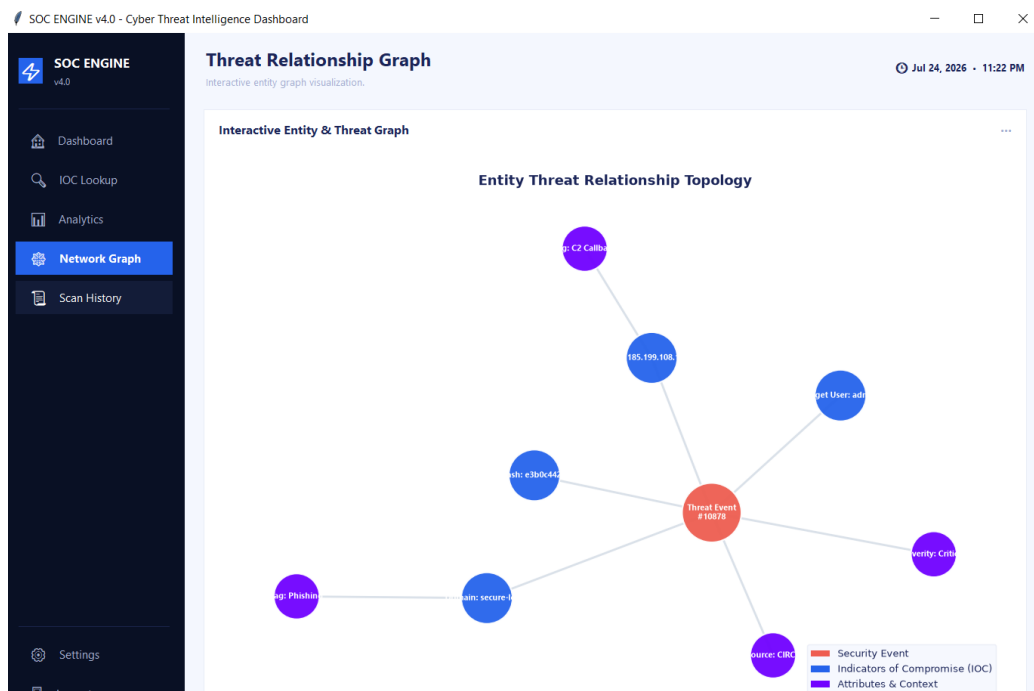
The analytics view below (Figure 12) shows an overview of all the scans mentioned above in the Risk Score histogram. Most result fall in three categories of low, medium( about 40) and high (nearly 100) risk, reflecting the three-tier categorization of risk scoring in SOC Engine.



*Figure 12: Risk score distribution histogram across recorded scans.*

## 5.4 Network Graph

Figure 13 below represents the network graph of the selected threat event, namely, threat event. The main attributes of the event include an IP address, domain, and hash value of the system's file, along with the target user, tags, source, and severity. The figure below support the analysts in visualizing the relationship of the identified threat.



*Figure 13: Interactive threat relationship graph for a single investigated event.*

## 5.5 Scan History and Audit Trail

Figure 14 below presents the Investigation History screen, which displays all completed scans. Investigation history lists the scan time, type of indicator of compromise (IOC), target, risk level, and analyst. The history is automatically updated every time an investigation lookup is performed, allowing an analyst to rely on it for future reference and incident investigation. Figure 14 shows completed scans history.

| ID | Timestamp           | Type | Target Value    | Risk Score | Analyst ID |
|----|---------------------|------|-----------------|------------|------------|
| 7  | 2026-07-24 17:25:51 | IP   | 185.199.108.153 | 41.0       | admin      |
| 6  | 2026-07-24 17:24:06 | IP   | 185.220.101.34  | 85.0       | admin      |
| 5  | 2026-07-24 17:22:59 | IP   | 107.170.42.79   | 0.0        | admin      |
| 4  | 2026-07-24 17:22:40 | IP   | 192.168.101.1   | 0.0        | admin      |
| 3  | 2026-07-22 11:00:38 | IP   | 185.199.108.153 | 40.0       | admin      |
| 2  | 2026-06-24 18:23:03 | IP   | 185.220.101.34  | 100.0      | admin      |
| 1  | 2026-06-24 18:22:32 | IP   | 192.168.101.1   | 0.0        | admin      |

Figure 14: Scan Audit Logs and History, showing the full record of past investigations.

## 5.6 Discussion

Overall the application achieved chapter 1 objectives. First, it enables one IOC lookup that is able to fuse the results from multiple threat intelligence sources. The results are consolidated in one risk score metric, which is easily understandable and automatically stored at the time of scans. Second, the dashboard and graph features allows for a better understanding of the results than in a table view. Its main drawback however is the sequential query of two APIs compared to concurrent query, for IOC lookups. These issues should be addressed in future development.

## CHAPTER 6: SECURITY CONSIDERATIONS

Any security tools is supposed to not be a burden for its users. Api credentials are never stored in plain Text, but instead are stored encrypted at rest using [Fernet](#) symmetric encryption ([Python Software Foundation, 2026](#)), with the key being also encrypted and stored separately from the encrypted contents. All of the database

queries that interact with user input are parameterized, thereby eliminating the possibility of SQL injection in the lookup and search fields.

There are some restrictions which are obvious and need to be mentioned straight out. Currently it is not using its own authentication, but it is fine if it is being used by one analyst, but not if being used by a team of analysts without having its own login and role system. Like the config.json file, the secret.key file can be vulnerable if the attacker has local access to it as well as to the config.json file, as the underlying API credentials can be retrieved. The key storage deferred to the operating system's credential vault is not indicated as a solved problem, but is listed as future work.

## CHAPTER 7: CONCLUSION AND FUTURE WORK

SOC Engine was built to solve a simple question: Can the “multi-tab” process of looking at an indicator in multiple places be reduce to a single click, and the result recorded automatically? The working system in Chapter 5 provides an answer to this question for the scope of interest it targets. It integrates VirusTotal and AbuseIPDB with a single risk score based on a partial approach to evaluates the risk, retains all lookups and displays the results in two different views: statistical and relational.

Several extensions are made possible by limitations encountered during the development: scoring file hashes and domains beyond IP addresses and roles for a multi-analyst audit trail, or integrating other feeds, such as [Shodan](#) for exposed services or an internal [MISP](#) for the company's own indicators, so that the same scoring and visualization pipeline can be fed with richer evidence. All of these do not involve rearchitecting the system, because the modularity of the separation of the UI, lookup engine, database, and API clients were created to facilitate the addition of each as a new module, not as a rewrite.

## REFERENCES

- AbuseIPDB. (2026). AbuseIPDB API Documentation. Marathon Studios Inc. Available at: <https://docs.abuseipdb.com/>
- Hagberg, A., Swart, P., & Chult, D. S. (2008). Exploring network structure, dynamics, and function using NetworkX. Proceedings of the 7th Python in Science Conference (SciPy 2008), 11-15. Available at: <https://proceedings.scipy.org/articles/TCWV9851>
- Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. Computing in Science & Engineering, 9(3), 90-95. Available at: <https://doi.org/10.1109/MCSE.2007.55>
- McKinney, W. (2010). Data structures for statistical computing in Python. Proceedings of the 9th Python in Science Conference, 56-61. Available at: <http://conference.scipy.org/proceedings/scipy2010/mckinney.html>
- MISP Project. (2026). MISP Threat Intelligence & Sharing — Documentation and Support. Available at: <https://www.misp-project.org/documentation/>
- Python Software Foundation. (2026). cryptography: Cryptographic recipes and primitives for Python. PyPI. Available at: <https://pypi.org/project/cryptography/>
- ReportLab. (2026). ReportLab PDF Toolkit Documentation. ReportLab Ltd. Available at: <https://www.reportlab.com/docs/reportlab-userguide.pdf>
- Shodan. (2026). Shodan Developer API Documentation. Available at: <https://developer.shodan.io/api>
- VirusTotal. (2026). VirusTotal API v3 Documentation. Google LLC. Available at: <https://docs.virustotal.com/reference/overview>

## **YouTube**

<https://www.youtube.com/watch?v=lY5fXUM5OZE>

## **Github**

<https://github.com/csah45564-debug/Introduction-to-Programming.git>